

HOSTED BY



ELSEVIER

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

An empirical assessment of ensemble methods and traditional machine learning techniques for web-based attack detection in industry 5.0

Oumaima Chakir^a, Abdeslam Rehaïmi^a, Yassine Sadqi^a, El Arbi Abdellaoui Alaoui^b, Moez Krichen^{c,d}, Gurjot Singh Gaba^{e,*}, Andrei Gurtov^e

^a Laboratory LIMATI, FPBM, USMS University, Beni Mellal, Morocco

^b IEVIA Team, IMAGE Laboratory, Department of Sciences, ENS, University of Moulay Ismail, Meknes, Morocco

^c Faculty of CSIT, Al-Baha University, Saudi Arabia

^d ReDCAD Laboratory, University of Sfax, Tunisia

^e School of Computer and Information Science, Linköping University, Sweden

ARTICLE INFO

Article history:

Received 31 December 2022

Revised 3 February 2023

Accepted 9 February 2023

Available online 15 February 2023

Keywords:

Cybersecurity

Ensemble methods

Industry 5.0

Machine learning

Web-based attack detection

ABSTRACT

Cybersecurity attacks that target software have become profitable and popular targets for cybercriminals who consciously take advantage of web-based vulnerabilities and execute attacks that might jeopardize essential industry 5.0 features. Several machine learning-based techniques have been developed in the literature to identify these types of assaults. In contrast to single classifiers, ensemble methods have not been evaluated empirically. To the best of our knowledge, this work is the first empirical evaluation of both homogeneous and heterogeneous ensemble approaches compared to single classifiers for web-based attack detection in industry 5.0, utilizing two of the most realistic public web-based attack datasets. The authors divided the experiment into three main phases: In the first phase, they evaluated the performance of five well-established supervised machine learning (ML) classifiers. In the second phase, they constructed a heterogeneous ensemble of the three best-performing ML algorithms using max voting and stacking methods. In the third phase, they used four well-known homogeneous ensembles to evaluate the performance of the bagging and boosting method. The results based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets revealed that bagging, particularly Random Forest, outperformed single classifiers in terms of accuracy, precision, F-value, FPR, and area of the ROC curve with values of 99.597%, 98.274%, 99.129%, 0.523%, 100 and 99.867%, 99.867%, 99.867%, 0.267%, 100, respectively. In contrast, single classifiers performed better than boosting and stacking. However, in terms of FPR, the boosting exceeded single classifiers. Max voting is appropriate when accuracy, precision, and FPR are the primary concerns, whereas single classifiers can be employed when recall, FNR, training, and prediction times are critical elements. In terms of training time, ensemble approaches are more likely to be affected by data volume than single classifiers. The paper's findings will help security researchers and practitioners identify the most efficient learning techniques for securing web applications.

© 2023 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Industry 5.0 integrates smart, linked devices with human cognitive and critical thinking to improve industrial efficiency and foster flexibility between humans and machines (Maddikunta et al., 2022; Zeb et al., 2022; Maddikunta et al., 2022). This next industrial revolution has the potential to improve the overall quality of production by delegating mundane and repetitive tasks to robots and machines while keeping those that need creative problem-solving for human workers. Moreover, industry 5.0 which encompasses more than just manufacturing, is based on the fourth industrial revolution (industry 4.0) (Gaba et al., 2020; Boopalan

* Corresponding author.

E-mail addresses: oumaima.chakirfpb@usms.ac.ma (O. Chakir), abdeslam.rehai-mi@usms.ac.ma (A. Rehaïmi), y.sadqi@usms.ma (Y. Sadqi), e.abdellaoui@umma.ac.ma (E.A. Abdellaoui Alaoui), moez.krichen@redcad.org (M. Krichen), gurjot.singh@liu.se (G.S. Gaba), andrei.gurtov@liu.se (A. Gurtov).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

et al., 2022; Masud et al., 2021) and is enabled by developments in information and communication technology (ICT) that include Artificial Intelligent (AI), automation, big data analytics, IoT, robotics, and edge computing. However, cybersecurity attacks are one of the critical challenges faced by industry 5.0 human-centric applications (Maddikunta et al., 2022; Leng et al., 2022; Kumar et al., 2022c). In fact, software, network, hardware, operating systems, firmware, and data security should all be taken into account while securing an industry 5.0-based system (Wu et al., 2018). Particularly, the software layer has become a lucrative and common target for hackers who purposefully exploit web-based threats and launch assaults that could compromise critical industry 5.0 functionalities such as confidentiality, integrity, and availability (Maddikunta et al., 2022; Wu et al., 2018; Alsaedi et al., 2022). To prevent these kinds of attacks, it is necessary to use security systems, such as Intrusion Detection Systems (IDS) (Sadqi and Mekkaoui, 2021; Khan et al., 2021) and Intrusion Prevention Systems (IPS) (Liao et al., 2013; Jemal et al., 2021). When an IDS or IPS is implemented in the context of protecting web applications, it is generally called a Web Application Firewall (WAF) (Desmet et al., 2006). In other words, WAFs can be considered as a special case of IDS or IPS that work at the OSI application layer, and are specialized in analyzing HTTP and HTTPS traffic with the goal of securing web applications, either by detecting or/and blocking web-related attacks (Sadqi and Maleh, 2022).

Signature-based WAFs, such as ModSecurity, are the most deployed type (Schmitt and Schinzel, 2012; Tian et al., 2019). These systems can effectively detect known attacks with a low false-positive rate (FPR), but they cannot detect unknown attacks, such as zero-day attacks. In contrast, anomaly-based systems perform better at detecting unknown attacks, but they generate a higher FPR. Thus, traditional signature and anomaly-based attack detection and prevention systems are inappropriate to guarantee the security of modern web applications (Jemal et al., 2021). With the development of AI techniques that are widely utilized in image and text recognition, a big part of cybersecurity research is focused on these methods since they appear to be effective at improving the performance of both signature-based and anomaly-based attack detection systems (Chan et al., 2013). Since 2011, techniques based on AI have received a remarkable increase in attention for web application security (Ghaffarian and Shahriari, 2017; Alaoui and Nfaoui, 2022). One of the most well-liked AI subsets is Machine Learning (ML)-based solutions (Asif et al., 2021; Kumar et al., 2022a; Kumar et al., 2022b). Specifically, ML methods are thought to be the most cutting-edge approach for attack detection systems. But traditional ML models often give high FPR and False Negative Rate (FNR) for cyberattacks that are becoming more complex (Gupta and Rani, 2020; Sommer and Paxson, 2010).

To solve the issue of traditional ML approaches, ensemble methods (also known as multi-classifier systems, committee-based learning, or classifier ensembles) are one of the most important directions in ML research for cyberattack detection (Tama and Lim, 2021; Aburomman and Reaz, 2017). The main idea behind using classifier ensembles is that the aggregating of the predictions from multiple individual classifiers gives better detection results (Sagi and Rokach, 2018; Abba et al., 2022). In addition, tying together the predictions of multiple classifiers would fix mistakes produced by each classifier and lead to better performance than a single classifier (Valentini and Masulli, 2002; Opitz and Maclin, 1999; Belouch and Hadaaj, 2017). According to Aburomman and Reaz (2017), there are two types of ensemble methods: (1) homogeneous ensemble that is built on base models of the same type, such as Random Forest (RF), and (2) heterogeneous ensemble that incorporates various types of base models, for example Support Vector Machine (SVM), K-Nearest Neighbors (KNN) and Decision

Tree (DT). The vast majority of multi-classifier systems are homogeneous ensembles (Tama and Lim, 2021).

Recently, ensemble methods have been widely used in various fields, including cybersecurity (Tama et al., 2020; Zhou et al., 2020; Zhou and Wang, 2019). Nevertheless, the need for an appropriate, high-performing ML algorithm in the cybersecurity field remains an unresolved research problem (Tama and Lim, 2021). Moreover, the majority of earlier research has placed a lot of emphasis on the performance of learning-based techniques for network-based attack detection (Sommer and Paxson, 2010; Tama and Lim, 2021; Aburomman and Reaz, 2017). In Caruana and Niculescu-Mizil (2006), demonstrated that the performance evaluation of learning-based techniques is application and implementation dependent. Hence, there is still a need for a benchmark of ensemble techniques and base classifiers in the context of the web-based attack detection. Despite the frequently use of accuracy as a primary evaluation parameter in related research, it is impossible to fairly compare and evaluate different ML-based solutions (Maseer et al., 2021).

To address the aforementioned shortcomings, this work evaluates the effectiveness of ensemble methods in web-based attack detection and comparing it to single classifiers using reliable evaluation metrics (Accuracy (A), Recall (R), Precision (P), F-value (F1), FPR, FNR, ROC curve, training and prediction time (TT and PT)). Particularly, the most used single learner algorithms and ensemble techniques were investigated, taking into account both homogeneous and heterogeneous forms of ensemble classifiers. Additionally, particular focus was placed on those learning methods that often yield effective attack detection results (Tama and Lim, 2021; Aburomman and Reaz, 2017; Maseer et al., 2021). To the best of our knowledge, this work is the first empirical assessment of both homogeneous and heterogeneous ensemble methods compared to single classifiers for web-based attack detection in industry 5.0, using two of the most used public and realistic web-based attack datasets. In fact, the limited number of available public datasets to evaluate web-based attack detection systems is one of the biggest problems in web-based attack detection (Oumaima et al., 2021; Tama et al., 2020). The availability of public datasets can assist web application administrators in identifying and classifying security issues, implementing effective countermeasures and defense strategies, and increasing situational awareness (Catillo et al., 2021). To construct an efficient machine learning-based web attack detection system, it is crucial to use a public dataset that includes online traffic and accurately represents actual web application attacks (Tama et al., 2020). An ML-based web attack detection system built on datasets that do not contain web traffic will be incapable of accurately detecting attacks targeting web applications. The most known and widely used datasets were generated for network-based attack detection may be inefficient to evaluate web-based attack detection solutions, because they do not contain real-world web attacks. That's why in this paper the authors decided to use ECML/PKDD 2007, CSIC HTTP 2010 open-source datasets, which are appropriate for web attack detection. Although the use of outdated public attack detection datasets has drawn criticism (Małowidzki et al., 2015), recent papers published in high-quality journals have used the ECML/PKDD 2007 and CSIC HTTP 2010 datasets (Shahid et al., 2022; Luo et al., 2020; Tekerek, 2021). Using the widely used public datasets is required for reliable performance comparison of the state-of-the-art ML methods (Yang et al., 2022).

Based on the principle that combining the predictions of multiple classifiers would correct the errors produced by each classifier and result in better performance. The majority of recent studies have focused on using ensemble methods rather than single classifiers to improve the detection performance of web-based attacks.

To the best of our knowledge, there are no studies that have proven the validity of this principle in the context of web security. Through an empirical evaluation of ensemble approaches and single classifiers for web-based attack detection in Industry 5.0, this study aims to fill this research gap. The following research questions (RQs) are defined to encompass the purpose and scope of this article:

- RQ1: Are the ensemble methods suitable for web-based attacks detection?
- RQ2: Which ensemble method is better suited for web-based attack detection?
- RQ3: How effective are ensemble-learning methods compared to single classification techniques for web-based attack detection?

The authors believe that answering these three core questions will help to assist academics and practitioners in the hard task of choosing the most appropriate algorithm for web-based attack detection. In the experiments, 5 well-established supervised ML classifiers are used with their variants, including SVM (Kumar et al., 2019), KNN (Kumar et al., 2019) Naive Bayes (NB) (Khan et al., 2021), DT (Kumar et al., 2012), and Logistic Regression (LR) (Kasongo and Yanxia, 2020), and their results suggested which classifiers to be integrated in the construction of a heterogeneous ensemble using Max voting (Lower and Zhan, 2020) and Stacking (Belouch and Hadaj, 2017; Lower and Zhan, 2020) methods. Next, the four well-known homogeneous ensemble RF, BaggingClassifier (BC), XGBoost, and Gradient boosting (GB) are evaluated using bagging (Belouch and Hadaj, 2017; Lower and Zhan, 2020; Abuomman and Reaz, 2016) and boosting (Belouch and Hadaj, 2017; Lower and Zhan, 2020; Schapire, 1999) techniques.

This paper is organized as follows, Section 2 describes related works. The dataset, classification methods, evaluation metrics are discussed in detail in Section 3. The experimental results are presented and discussed in Section 4. The challenges of the current proposed ML-based Web Security System (ML-WSS) are presented in Section 5. Section 6 is dedicated to the conclusion and future works.

2. Related work

In this section, the authors highlight the recent notable works that have used single classifiers and ensemble methods for web-based attack detection. Moreover, papers that focused on performance comparison for ensemble classifiers and single classifiers in cyberattack detection are presented.

2.1. Web-based attack detection using single classifiers

The authors in Kozik et al. (2015) compared the performance of three algorithms: J48, AdaBoost, and Naive Bayes (NB). J48 had a detection rate of 95.97%, while NB had an 88.89% rate and AdaBoost had an 83.23% rate. The authors believed that the information about the accuracy metric is sufficient to demonstrate the effectiveness of the ML algorithms. In contrast, other performance parameters discussed in this paper are P, R, F1, FPR, FNR, ROC curve, TT, and PT. Using the CSIC HTTP 2010 dataset, the authors in Smitha et al. (2019) evaluated the performance of SVM, Logistic Regression (LR), DT, and Neural Network (NN) algorithms in detecting web-based attacks. Their experiment results revealed that LR outperformed other in terms of A, R, P, F1, and ROC curve with values of 97%, 95%, 96%, and 97%, respectively. However, in terms of P, SVM performed better with a value of 94%. The authors in Khan et al. (2017) investigated the performance of KNN, NB, J48,

and SVM classifiers in detecting Cross Site-Scripting (XSS)-based attacks using a customized dataset of 1924 cases, 1515 benign and 409 JavaScript codes as harmful instances. According to the results of their experiments, all four classifiers achieved accuracy values of 97.14%, 95.06%, 99.22%, and 94.55%, respectively. The authors investigated the performance of KNN, SVM using polynomial and linear kernel, and RF classifiers in detecting XSS attacks in their paper (Mereani and Howe, 2018). The experiments were carried out with the help of a customized training data set of 2,000 malicious and benign scripts, as well as a test data set of 13,000 malicious and benign scripts. All of these classifiers performed well in terms of A, P, R, and TNR, according to their results.

2.2. Web-based attack detection using ensemble methods

In Tama et al. (2020), the authors proposed a new method for anomaly-based attack detection systems for securing web applications using stacking ensemble. In contrast to traditional stacking ensembles that suggest the use of weak base models, the authors proposed the combination of multiple strong models which are RF, Gradient Boosting Machine (GBM), and XGBoost. Their experimental results based on CSIC 2010 V2, CIC-IDS 2017, NSL-KDD, and UNSW-NB15 datasets showed better detection performance in terms of A, 98.82%, 99.99%, 92.17%, 92.45%, respectively. To improve the detection ability of the existing IDS, the authors in Zhou et al. (2020) proposed a novel IDS which is based on correlation feature selection (CFS) for selecting the good features and an ensemble method that combines the decision from three classifiers C4.5, RF, and Forest by penalizing attributes (Forest PA) into one final decision by using a vote classifier. Their experimental results based on NSL-KDD, AWID, and CIC-IDS 2017 datasets achieved higher A with values of 99.8%, 99.5%, and 99.8%, respectively. In Zhou and Wang (2019), the authors proposed a new XSS attack detection approach based on ensemble methods, were used the Bayesian networks (BN) as base models and max voting methods as the final decision function. The dataset used in was collected from GitHub and other security forums and it included 16151 XSS payloads and 135507 normal for training and 3497 XSS payloads, and 6503 normal for testing. To model real-world attack scenarios, they tested the accuracy of their approach under the different percentages of malicious sampled in the dataset. Their method achieved better accuracy 98.54% even when the malicious sampled accounted for the majority of the testing dataset (45% of malicious records percentage). In Lower and Zhan (2020), the authors presented a study on the effectiveness of ensemble methods for cybersecurity using the NSL-KDD dataset. Specifically, they tested three types of these methods, voting (they evaluated both hard and soft voting), bagging, and boosting. For voting methods, the authors (Lower and Zhan, 2020) used in their experiment, KNN, DT, MLP classifier and LR as base models. They divided the experiment into three steps: they started by combining only two models DT and KNN, next, they added the MLP classifier. In the last step, they combined the four selected algorithms as base models and compared the results obtained in each step. For bagging and boosting methods, the authors used RF and AdaBoost classifier, respectively, with n decision trees as base models. They progressively increased the number of DT in order to see if it will improve the overall performance of the classifier. Their results showed that, compared to the voting method, RF and AdaBoost achieved better performance with an accuracy of more than 80.22%. In Tripathy et al. (2020), the authors proposed a study of the effectiveness of using ML algorithms for securing web applications deployed on cloud Software as a Service (SaaS) against SQL injection (SQLI) attacks, which are one of the critical SaaS vulnerabilities that enable attackers to steal confidential information's. RF, AdaBoost, Stochastic Gradient Descent (SGD), Deep Artificial Neural Network

(ANN), Tensorflow's linear classifier, and BosstedTree Classifiers were all chosen to the test. According to their results, all the classifiers had an accuracy higher than 98%, but RF outperformed them all with a score of 99.8% accuracy.

2.3. Performance comparison of ensemble learning methods and single classifiers

In the literature, there are limited papers that focused on performance comparison for ensemble and single classifiers ML-based solutions for attack detection. In [Tama and Lim \(2021\)](#), the authors incorporate a comparison of the ensemble's basic classifiers and the Stack of Ensemble (SoE). A variety of homogeneous ensembles, including RF, XGBoost, and GBM, are combined by SoE architecture. In addition, random search is used to find the optimal learning parameters for each base classification. The authors in [Tama and Lim \(2021\)](#) validated previous research results related to the significant performance improvement of ensemble methods in comparison to single classifiers approaches for network-based attack detection. However, as it depends on a number of variables, including base classifiers, ensemble methods category, etc., this is not always the case. By assessing the true positive and negative rates, A, R, P, F1, TT, and PT of 10 supervised and unsupervised learning algorithms, including KNN, DT, RF, SVM, NB, ANN, Convolutional Neural Network (CNN), Expectation–Maximization (EM), Self-Organizing Maps (SOM) and k-Means, the work in [Maseer et al. \(2021\)](#) have assessed the effectiveness of anomalous attack detection. The kNN, DT, and NB algorithms, according to the scholars, are better able to identify web attacks than the other tested algorithms. However, the paper did not take into consideration ensemble methods. The work in [Tama and Rhee \(2017\)](#), focused on the assessment of ensemble learning in the context of network-based attack detection using five base classifiers, including RF, DT, LR, Functional Trees (FT) and CART. However, their study did not include homogeneous ensemble evaluation.

[Table 1](#) presents a comparison between the presented related works and this paper. Comparing all the works presented in [Table 1](#), the most similar to this study is [Tama and Rhee \(2017\)](#). What distinguishes our work from [Tama and Rhee \(2017\)](#) is that we evaluated the effectiveness of both homogeneous and heterogeneous ensembles for web-based attack detection in industry 5.0 and compared their performance to that of a single classifier to determine whether they consistently outperform single classifiers, as is common. Unlike [Tama and Rhee \(2017\)](#), which only used A, P, R, and F1, the authors of this paper used FPR, FNR, TT, PT, and ROC curve metrics to evaluate the performance of learning algorithms.

3. Research methodology

In this section, the authors describes the proposed benchmarking methodology. Afterwards, web attack datasets, data preprocessing and classification algorithms utilized in experiments are presented. Finally, the performance metrics are introduced.

3.1. Overview

The main objective of this work is to compare the efficacy of ensemble methods to a single classifier in the web-based attack detection domain, using well-established public datasets: ECML/PKDD 2007 and CSIC HTTP 2010. The experiment is divided into the three phases:

- Phase 1: The authors compare the performance of the most used ML algorithms for web-based attack detection tasks in industry 5.0, which are KNN, DT, SVM, NB, and LR.
- Phase 2: The authors create a heterogeneous ensemble of the three best-performing ML algorithms in terms of the evaluation metrics using max voting and stacking methods.
- Phase 3: The authors use RF, BC, GB, and XGBoost, which are all homogeneous ensemble methods, to evaluate the performance of bagging and boosting methods.

3.2. Benchmark datasets

The current widely used standard public datasets for attack detection benchmarking such as NSL-KDD and KDD Cup 99 are not appropriate for web-based attack detection evaluation, because they do not include web-based attacks ([Oumaima et al., 2021](#)). For that reason, two datasets ECML/PKDD 2007 and CSIC HTTP 2010 ([Web Application attacks datasets, 2022](#)), which are created specifically for web traffic anomaly identification, are selected. In the following, the two utilized datasets are briefly introduced.

3.2.1. ECML/PKDD 2007

A dataset that was generated for ECML/PKDD Discovery Challenge. This dataset contains 24504 and 10502 valid requests for training and testing, respectively and 15110 malicious requests for testing. It includes various types of web-based attacks like XSS, SQLI, LDAP Injection, XPATH Injection, Path traversal, Command execution, and SSI attacks ([Web Application attacks datasets, 2022](#); [Raïssi et al., 2007](#)).

3.2.2. CSIC HTTP 2010

A dataset that contains real requests generated to an e-commerce application. This dataset contains two types of requests, 36000 normal requests for training, 36000 normal for testing, and 25000 requests classified as attacks for testing. It contains various types of web attacks such as SQLI, XSS, CRLF Injection, and Buffer overflow ([Web Application attacks datasets, 2022](#); [Giménez et al., 2010](#)). The main problem with the ECML/PKDD 2007 and CSIC HTTP 2010 datasets is that they contain only valid training requests ([Oumaima et al., 2021](#); [Web Application attacks datasets, 2022](#)). The training phase is the most crucial stage in constructing learning algorithms, during which the algorithms learn to identify normal activity and assault patterns. An ML-based attack detection system trained with only legitimate data will not be able to detect attacks that mimic normal activity ([Mokhtari et al., 2021](#)). Therefore, to build an efficient machine learning-based attack detection system, it is necessary to train the system with both legitimate and malicious data. Consequently, the authors took the original datasets and turned them into personalized datasets by extracting attack samples from the testing dataset and using them for training. The datasets size used in this work is presented in [Table 2](#).

3.3. Data preprocessing

As illustrated in [Fig. 1](#), the authors started the experiment by extracting the data from ECML/PKDD 2007 and CSIC HTTP 2010 datasets using the urlparse library. The authors focus mainly on extracting both legitimate and malicious data from.

Algorithm 1. Proposed Tokenization Algorithm

operate with numerical characteristics, the authors encoded each token to a vector using the CountVectorizer tool (Tama et al.,

Algorithm 1 : Proposed Tokenization Algorithm

```

Input{data}
Variables
/*Load key signatures of each attack type*/
BagWords1=[signature_list1]
BagWords2=[signature_list2]
FinalBag=[]
BEGIN Algorithm
/* Decode the url and replace characters like '%xx' with a single character*/
unquoted_data = urllib.parse.unquote(data)
/*Put the url in lowercase*/
lowercase_data = unquoted_data.lower()
/*Split by '&' */
TokensOne [] = split lowercase_data to a list by the special character('&')
for each word in TokensOne do
  /* Split by '=' */
  TokensTwo [] = SPLIT each word to a list by the special character('=')
  /* Split by selecting the occurring words defined in BagWords1 */
  Signs =find_signes(word, BagWords1)
  if Signs is not empty then
    | FILL FinalBag WITH Signs
  end
  for each new_word in TokensTwo do
    | FILL TokensTree WITH new_word
  end
end
for each word in TokensTree do
  | TokensFour [] = split each word to a list by the special character('space','+','—')
end
for each word in TokensFour do
  /* Split by selecting the occurring words defined in BagWords2 */
  SignsFound = find_signes(word, BagWords2)
  if SignsFound is not empty then
    | APPENDS SignsFound TO TokensFour
  end
end
Final_List = TokensFour
Remove empty values from the Final_List
Replace with null digits or the word containing digits exist in Final_List
Remove empty values from the list
/* Merge all tokens with FinalBag*/
Final_Tokens = MERGE Final_List WITH FinalBag
Output{Final_Tokens}
END Algorithm

```

GET, POST, and PUT requests. The authors extracted the training and testing data after cleaning the data by removing all null and duplicated data. Since this is a binary classification problem, the authors converted the string labels to numeric values. The class with a value of 0 for legitimate and 1 for malicious data. Then, the authors created a tokenization algorithm (see Algorithm 1) to transform the input string data into a list of tokens through many steps, firstly the authors decoded the input data using the same library urlparse to replace URL characters like '%xx' into its original character like '<', '>', ':' the authors have chosen to preserve those characters since several attacks, such as XSS attack, use them as payload construction. To keep the same format, the authors have converted each character to a lower case. Then, split the data using special characters such as '&', 'white space', '=', and '—'. Moving on, the authors have collected several keywords used to split data in Tripathy et al. (2020) and their own keywords gathered by analyzing attack types and the content of the dataset to make their own bag of words that are used for tokenization. Since all algorithms

2020). In this study, the authors evaluated the efficacy of ML algorithms using their default parameters to identify their performance in the default scenario. In order to categorize the data into malicious and legitimate, the authors supplied the training data to the classifiers. Finally, the authors evaluated the predictive models created by the classifiers with test data.

3.4. Evaluation metrics

To assess the performance of single and ensemble learning classifiers, the authors selected the most common metrics for web-based attack detection (Tama et al., 2020; Singh et al., 2022; Hasan et al., 2022) namely accuracy (A), recall (R), precision (P), F-value (F), False positive rate (FPR), False negative rate (FNR), training time (TT), prediction time (PT), and ROC curve.

- Accuracy (A): Refers to the ratio between the number of correctly detected attacks and the total number of attacks.

Table 1

Comparison between the presented related works and this paper.

Ref	Ensemble Methods	Algorithm	Evaluation Metrics	Dataset	Description	Limitations
(Tama and Lim, 2021)	Bagging, Boosting	RF,GBM, XGboost	A,FPR AUC	CIC IDS 2017, NSL-KDD, UNSW-NB15, UNSW 2018	The authors evaluated the performance of homogeneous ensembles in comparison with individual classifiers in network-based attack detection. Their experiments revealed that ensemble methods' performance depends on various factors, such as base classifiers.	Focuses only on network-based attacks and homogeneous ensembles and ignores P, R, FNR, TT, and PT metrics.
(Tama et al., 2020)	Stacking	RF,GB, XGboost	A, and FPR	CSIC 2010 V2, CIC IDS 2017, NSL-KDD, UNSW-NB15	The authors proposed a new IDS for web app security using the stacking method. According to the CIC-IDS 2017 dataset results, the proposed methods achieved a better performance in terms of A and FPR, with values of 99.99% and 0.46%.	Uses only A and FPR metrics, network-based attacks datasets (NSL-KDD and UNSW-NB15) and only focuses on the stacking method's performance.
(Zhou et al., 2020)	Voting	C4.5, RF, Forest PA	A, R, P, F1, and FPR	NSL-KDD, AWID, CIC-IDS 2017	The authors proposed a novel IDS based on a correlation feature selection and a voting method.	Uses only A, R, P and FPR metrics, network-based attacks datasets (NSL-KDD) and only focuses on the voting method's performance.
(Zhou and Wang, 2019)	Voting	BN	A	GitHub, Security Forums	The authors proposed a novel approach for XSS-based attack detection using the voting method. The proposed method achieved an A of 99.54%.	Uses only A metrics, only focuses on XSS-based attacks and the voting method's performance.
(Maseer et al., 2021)	-	KNN, NB, SVM RF, ANN, CNN EM, SOM, k-means	A, P,R F1, TT, DT	CIC IDS 2017	The authors evaluated the performance of the mentioned algorithms in anomalous attack detection. According to their results, the kNN, DT, and NB algorithms performed better in web attack detection than others.	Focuses only on the single classifier's performance and uses only 701 web attack instances, and ignores FPR, FNR, and ROC metrics.
(Lower and Zhan, 2020)	Voting, Bagging, Boosting	KNN, MLP, DT,LR, RF, AdaBoost	A	NSL-KDD	The authors evaluated the effectiveness of ensemble methods for cybersecurity. Their results showed that RF and AdaBoost achieved an A of more than 80.22%.	Uses only A to evaluate the proposed system's performance, has a low A value, only focus on homogeneous ensembles and voting method.
(Kozik et al., 2015)	Boosting	J48, NB, AdaBoost	A	CSIC HTTP 2010	The authors evaluated the performance of the mentioned algorithms in detecting web-based attacks. J48 achieved the highest A with a value of 95.97%.	Uses only A, the boosting method, and the single classifier's performance.
(Smitha et al., 2019)	-	SVM, LR, DT, NN	A, P, R, F1, ROC Curve	CSIC HTTP 2010	The authors evaluated the performance of the mentioned algorithms in detecting web-based attacks. LR achieved the highest performance with A, R, P, F1, and AUC values of 97%, 92%, 95%, 96%, and 97%, respectively.	Ignores FPR, FNR, TT, and PT metrics and only focuses on the single classifier's performance.
(Khan et al., 2017)	-	KNN, NB, J48, SVM	A, R, P, F1, FPR, ROC Curve	LeakiEst Example	The authors evaluated the performance of the mentioned algorithms in detecting XSS attacks. J48 achieved the highest A with a value of 99.22%.	Focuses only on XSS-based attacks and the single classifier's performance, and ignores FNR, TT, and PT metrics.
(Mereani and Howe, 2018)	-	KNN, SVML, RF, SVMF	A, R, P, and TNR	XSSed Archive	The authors examined the performance of the mentioned algorithms in detecting XSS attacks. KNN achieved the highest performance in terms of A, P, R, and TNR with values of 99.75%, 99.88%, 99.61%, and 99.88%.	Focuses only on XSS-based attacks and the single classifier's performance and ignores FNR, TT, PT, and ROC metrics.
(Tripathy et al., 2020)	Bagging, Boosting	RF, AdaBoost, SGD, Deep ANN, BosstedTree, and Tensorflow's Linear	A	Payload	The authors evaluated the performance of the mentioned algorithms in detecting SQLI attacks. Based on their results, the RF was the most accurate with a value of 99.8%.	Uses only A metrics, focuses on SQLI-based attacks and homogeneous ensembles.
(Tama and Rhee, 2017)	Voting, Stacking, Bagging, Boosting	RF, DT, LR FT, CART	A, P, R, F1	NSL-KDD, GPRS-WEP/WPA, GPRS-WPA2	The authors compare the effectiveness of ensemble learning methods to that of a single classifier in network-based attack detection	Focuses only on network-based attacks, heterogeneous ensemble and ignores FPR, FNR, TT, PT, and ROC metrics.
Proposed work	Voting, Stacking, Bagging, Boosting	MNB, BNB, KNN, SVML, SVMR, SVMs, LR, DT, RF, XGBoost, GB, BC	A, R, P,F1, FPR, FNR, ROC curve, TT, and PT	CSIC HTTP 2010 ECML/ PKDD 2007	The authors compare the effectiveness of homogeneous and heterogeneous ensemble learning to that of a single classifier in web-based attack detection.	Focuses only on traditional web-based attack detection.

Table 2

The datasets size used in this work.

Class	ECML/PKDD 2007		CSIC HTTP 2010	
	Train data	Test data	Train data	Test data
Normal	19649	8420	4000	1500
Attack	12510	2505	4000	1500

$$A = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- Recall (R): Calculate the number of correctly detected attacks.

$$R = \frac{TP}{TP + FN} \quad (2)$$

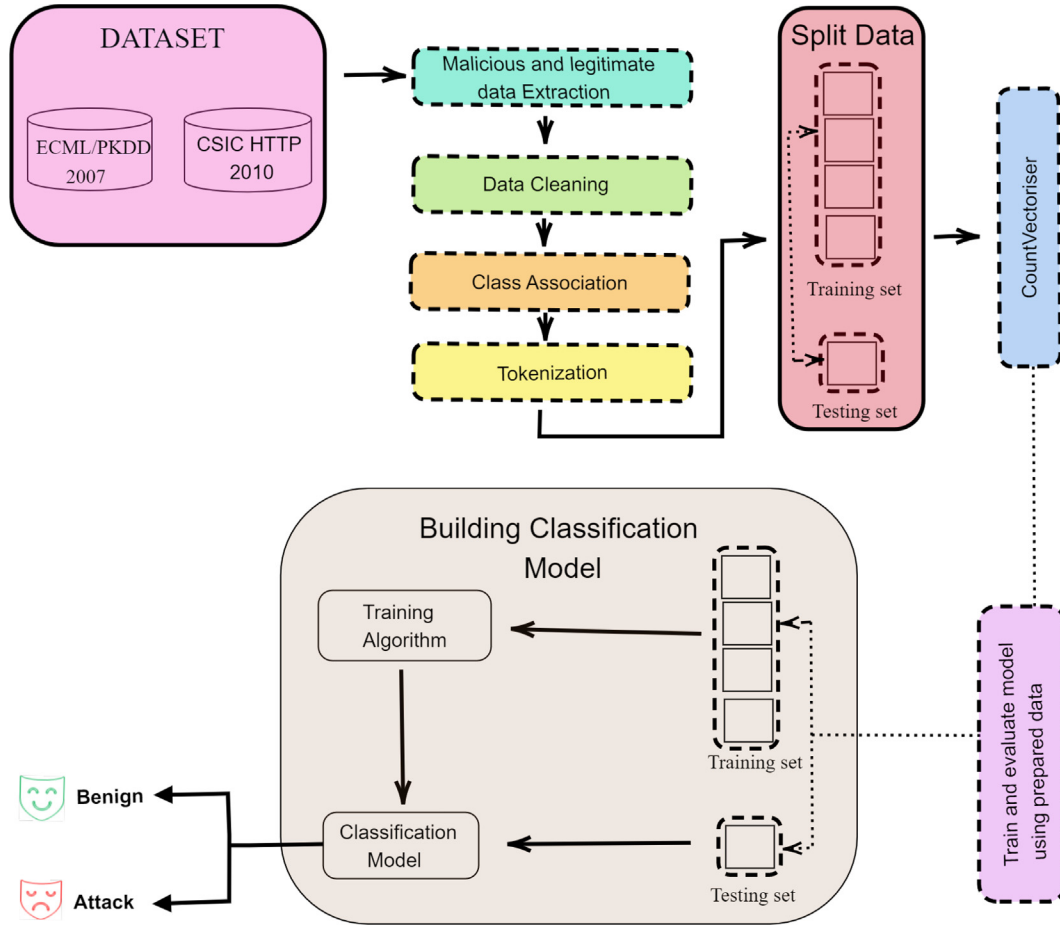


Fig. 1. Proposed benchmarking methodology.

- Precision (P): This metric is used to calculate the number of correctly classified legitimate requests.

$$P = \frac{TP}{TP + FP} \quad (3)$$

- F-value (F1): The harmonic average of recall and precision.

$$F1 = \frac{R \cdot P}{R + P} \quad (4)$$

- FPR: Calculate the number of legitimate data detected as an attack by the classifier.

$$FPR = \frac{FP}{FP + TN} \quad (5)$$

- FNR: Calculate the number of malicious data detected as legitimate data.

$$FNR = \frac{FN}{TP + FN} \quad (6)$$

- Training time (TT): Represents the amount of time required to train the whole dataset and create the learning model with the greatest fit.

$$TT = end_{time}^{training} - start_{time}^{training} \quad (7)$$

- Detection Time (DT): Measures how long it takes a learning algorithm to classify the entire testing data as an attack or normal.

$$DT = end_{time}^{testing} - start_{time}^{testing} \quad (8)$$

- ROC Curve: A graph that depicts the False Positive and True Positive Rates at different classification thresholds. It is a useful strategy for determining the best classification model. A high ROC value implies that a model can detect attacks, whereas a low number shows inefficiency.

4. Results and discussion

4.1. Experiment environment

The experiments presented in this paper were carried out using a hardware environment equipped with Intel(R) Core (TM) i5-3320 M*64 based CPU running at 2.6GHZ with 12G RAM installed and a 64-bit Windows 10 as the operating system. Table 3 presents the various tools used in this work.

4.2. Results

The evaluation metrics such as A, R, P, F1, FPR, and FNR are measured based on the confusion matrix generated during the testing process. TP and TN values indicate the number of attacks and normal data correctly classified by the classifier, while FN and FP val-

Table 3
Tools used in this work.

Tool	Conda	Sklearn	Jupyterlab	Python
Version	4.11.0	0.24.2	3.2.1	3.9.7

ues indicate the number of attacks and normal data incorrectly classified. As can be seen from Table 4, DT is more accurate in detecting web-based attacks than other classifiers. As shown in Table 5, the results based on the ECML/PKDD 2007 dataset demonstrated that SVML, DT, and LR classifiers performed better than others in terms of A and F1. Given the P and FPR of each classifier, BNB, KNN, and LR outperformed the other classifiers. However, it can be observed that SVML, DT, and MNB beat other single classifiers in terms of R and FNR. In contrast, the CSIC HTTP 2010 results showed that SVML, DT, and LR classifiers achieved the highest performance, compared to others, in terms of A, R, P, F1, and FNR. However, in terms of FPR it can be observed that SVMR performed better than others. During this study, the TT and PT also were measured because they are crucial assessment criteria for any web application security system. As can be seen from Fig. 2, the results based on the ECML/PKDD 2007 dataset revealed that MNB, BNB, KNN, LR and DT classifiers have the least TT and PT compared to others with values of (0.024s, 0), (0.023s, 0.024s), (0.007s,

8.321s), (4.211s, 0.001s), and (55.655s, 0.084s), respectively. Likewise, the results based on the CSIC HTTP 2010 dataset showed that MNB, BNB, KNN, DT, and LR classifiers needed the least time to train their classification model and predict the unseen data compared to others with values of (0.007s, 0.008s), (0,0), (0, 1.273s), (1.101s, 0), and (0.813s, 0), respectively. Fig. 3 depicts the ROC curve of each classifier based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets. As can be seen from this figure, SVML, DT, and LR classifiers performed better than others, with an area of the ROC values of more than 96%. In contrast, the KNN classifier achieved the lowest score of 89%, which prove the inefficacy of this classifier in detecting web-based attacks. The experimental results based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets demonstrated that SVML, DT, and LR classifiers have hit the best rates in term of most of the evaluation metrics compared to other classifiers. As a result, these classifiers will serve as base models for the remainder of this paper. Table 6 presents the confusion matrix of the heterogeneous ensemble based on the ECML/

Table 4
Confusion matrix of each single classifier.

Classifier	ECML/PKDD 2007				CSIC HTTP 2010			
	TN	FP	FN	TP	TN	FP	FN	TP
MNB	7862	558	45	2460	1436	64	122	1378
BNB	8419	1	428	2077	1429	71	136	1364
KNN	8412	8	526	1979	1424	76	265	1235
SVML	8346	74	3	2502	1406	94	14	1486
SVMR	8356	64	290	2215	1497	3	230	1270
SVMS	8330	90	371	2134	1315	185	494	1006
DT	8325	95	0	2505	1446	54	0	1500
LR	8369	51	192	2313	1485	15	70	1430

Table 5
The performance evaluation of each classifier based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets.

Classifier	ECML/PKDD 2007						CSIC HTTP 2010					
	A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)	A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)
MNB	94.481	98.204	81.511	89.082	6.627	1.796	93.800	93.800	93.866	93.798	4.267	8.133
BNB	96.073	82.914	99.952	90.639	0.012	17.086	93.100	93.100	93.181	93.097	4.733	9.067
KNN	95.112	79.002	99.597	88.112	0.095	20.998	88.633	88.633	89.257	88.588	5.067	17.667
SVML	99.295	99.880	97.127	98.485	0.879	0.120	96.400	96.400	96.532	96.397	6.267	0.933
SVMR	96.760	88.423	97.192	92.600	0.760	11.577	92.233	92.233	93.233	92.189	0.200	15.333
SVMS	95.780	85.190	95.953	90.252	1.069	14.810	77.367	77.367	78.579	77.124	12.333	32.933
DT	99.140	100	96.383	98.158	1.116	0	98.233	98.233	98.294	98.233	3.533	0
LR	97.776	92.335	97.843	95.009	0.606	7.665	97.167	97.167	97.230	97.166	1	4.667

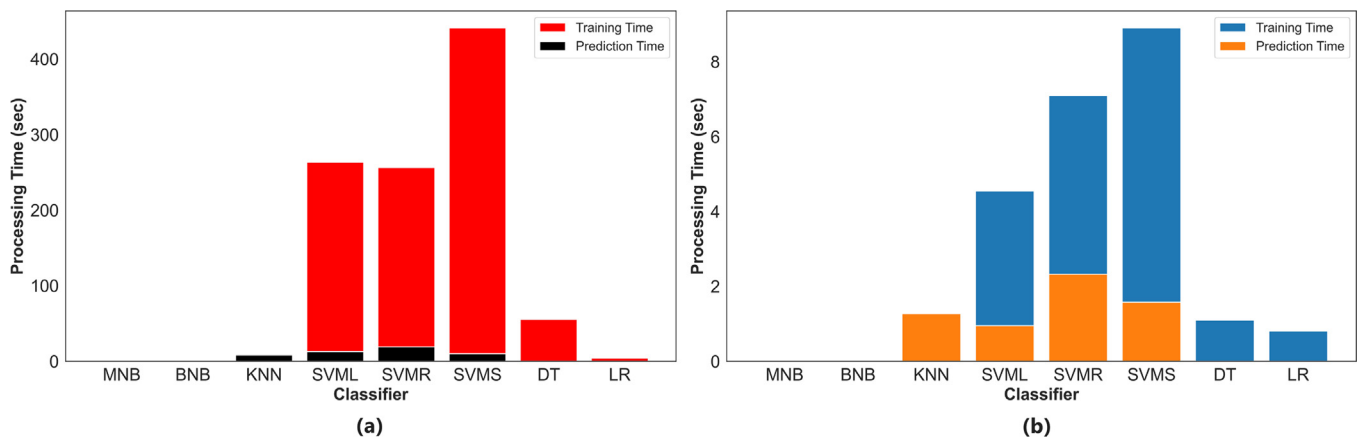


Fig. 2. Single classifier training and prediction time measurements based on (a) the ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

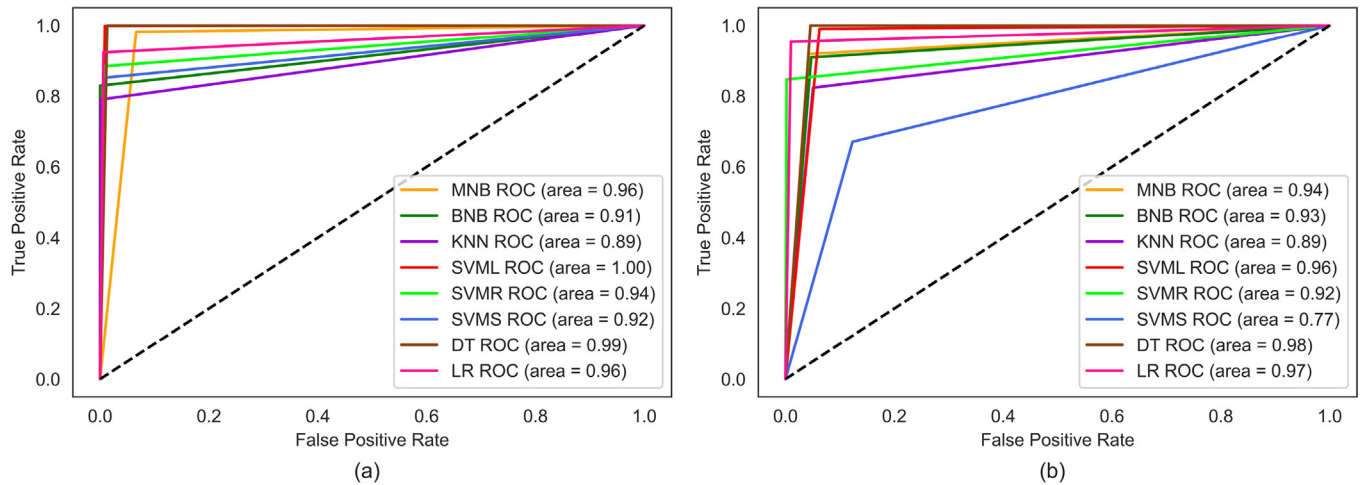


Fig. 3. ROC curve of each single classifier based on (a) the ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

Table 6

Confusion matrix of the heterogeneous ensemble.

Ensemble Methods	FE	ECML/PKDD 2007				CSIC HTTP 2010			
		TN	FP	FN	TP	TN	FP	FN	TP
MaxVoting	-	8363	57	3	2502	1464	36	13	1487
Staking	SVML	8280	140	0	2505	1435	65	0	1500
Stacking	DT	7423	997	51	2454	1240	260	7	1493
Stacking	LR	8310	110	27	2478	1367	133	11	1489

PKDD 2007 and CSIC HTTP 2010 datasets. Based on the results shown in Table 6, the Max voting approach classifies normal data more precisely than the stacking method. Among the 8420 valid data, 8363 were classified correctly, while only 57 were wrongly classified. Similarly, the CSIC HTTP 2010 results revealed that, out of 1500 valid data, 1464 were correctly identified by the Max voting method, while 36 were incorrectly classified. Unlike Max voting, the stacking approach with SVML as the final estimator (FE) is more accurate at identifying attacks.

According to the results shown in Table 7, Max voting outperforms the stacking method in terms of A, R, P, F1, FPR, and FNR. Also, the FE used has an impact on the stacking method's performance. Compared to others, when the authors employed SVML and LR classifiers as FE they reached better performance in terms of A, R, P, F1, FPR, and FNR. As shown in Fig. 4, the results based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets showed that the Max voting method required fewer TT and PT than the stacking method with values of (389.77s, 14.09s) and (6.773s, 1.030s), respectively. According to the ROC curves presented in Fig. 5, Max voting and Stacking using SVML as the FE are efficient for web-based attacks detection. Table 8 displays the confusion matrix of homogeneous ensembles. In contrast to boosting-based classi-

fiers, bagging-based classifiers are more accurate at detecting web-based attacks. As can be seen from Table 8, RF correctly classified all attacks. When comparing the performance of boosting-based classifiers, the authors observed that the XGBoost classifier surpassed GB in terms of A, R, P, F1, FPR, and FNR (see Table 9). Regarding the performance of bagging-based classifiers, RF outperformed others in terms of A, R, P, F1, FPR, and FNR (see Table 9). As shown in Fig. 6, the results based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets demonstrated that boosting-based classifiers need the least amount of time to train their classification models and make predictions based on the new data. On the other hand, the BC that used SVML as the base model (BM) took the longest amount of time for both the training and the prediction processes. Unlike other classifiers, the XGBoost classifier required fewer TT and PT with values of (5.656s, 0.043s) and (1.361s, 0.015s), respectively. As can be seen from Fig. 7, the RF classifier has the greatest value of the area of the ROC curve with a value of 100%, proving its capacity in detecting web-based assaults. In the following table, the authors present the strengths and weaknesses of single classifiers, heterogeneous ensembles, and homogeneous ensembles based on the experimental results of both the ECML/PKDD 2007 and the CSIC HTTP 2010 datasets (see Table 10).

Table 7

The performance of heterogeneous ensemble based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets.

Ensemble Methods	FE	ECML/PKDD 2007						CSIC HTTP 2010					
		A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)	A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)
MaxVoting	-	99.451	99.880	97.773	98.815	0.677	0.120	98.367	98.367	98.378	98.367	2.400	0.867
Stacking	SVML	98.719	100	94.707	97.282	1.663	0	97.833	97.833	97.923	97.832	4.333	0
Stacking	DT	90.362	98.523	70.838	82.418	12.067	1.477	91.100	91.100	92.303	91.036	17.333	0.467
Stacking	LR	98.746	98.922	95.750	97.310	1.306	1.078	95.200	95.200	95.501	95.192	8.867	0.733

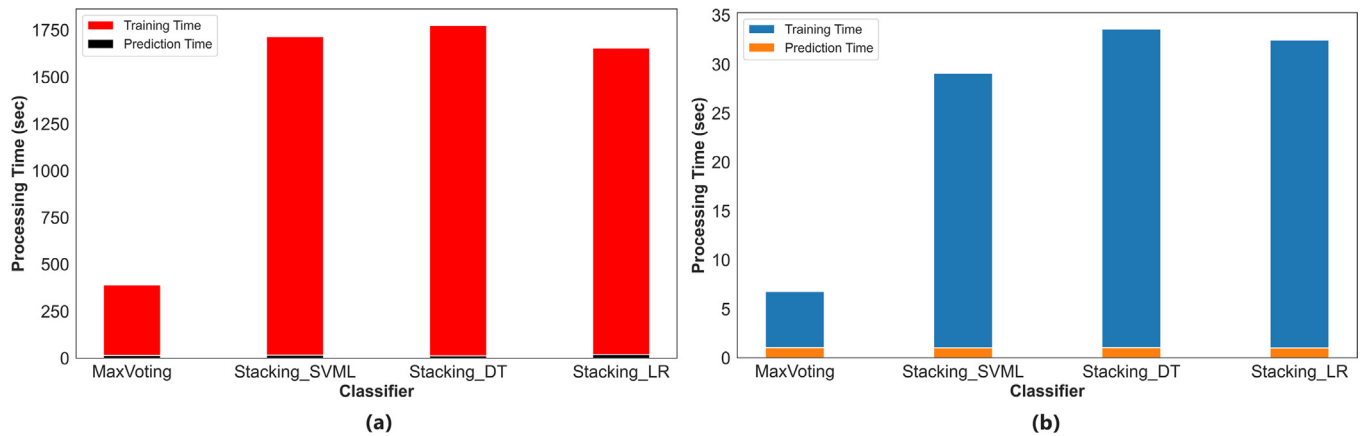


Fig. 4. Heterogeneous ensemble training and prediction time measurements based on (a) the ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

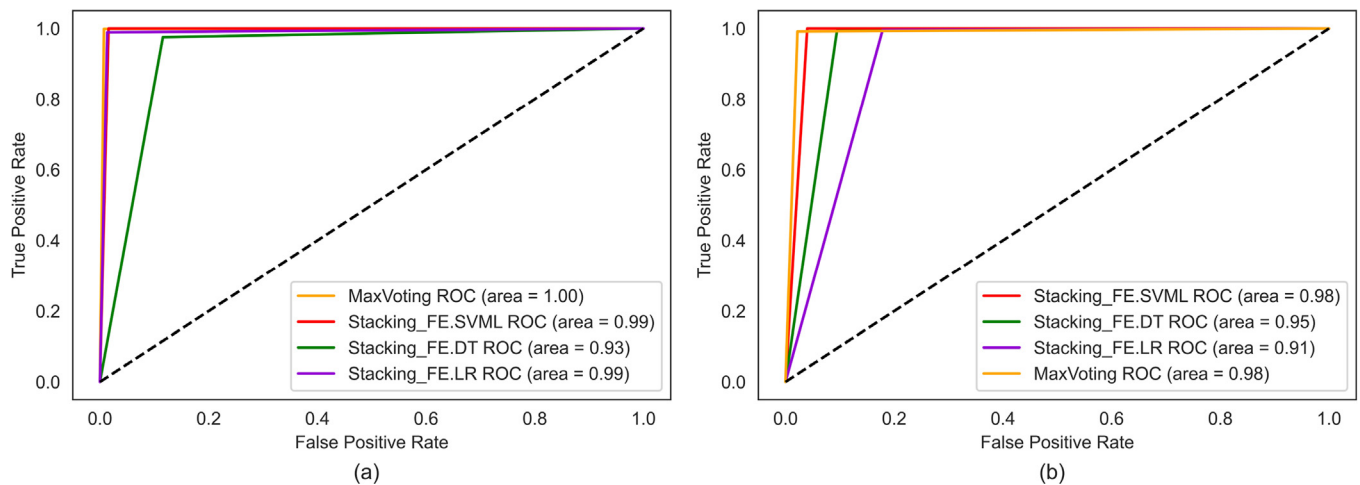


Fig. 5. ROC curve of heterogeneous ensemble based on the (a) ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

Table 8
Confusion matrix of homogeneous ensembles.

Ensemble Methods	Base Model	ECML/PKDD 2007				CSIC HTTP 2010			
		TN	FP	FN	TP	TN	FP	FN	TP
XGBoost	-	8374	46	366	2139	1497	3	233	1267
GB	-	8373	47	560	1945	1497	3	411	1089
RF	-	8376	44	0	2505	1496	4	0	1500
BC	SVML	8352	68	109	2396	1441	15	46	1454
BC	DT	8377	43	106	2399	1485	59	52	1448
BC	LR	8367	53	303	2202	1487	13	131	1369

Table 9
The performance of homogeneous ensemble based on the ECML/PKDD 2007 and CSIC HTTP 2010 datasets.

Ensemble Methods	BM	ECML/PKDD 2007						CSIC HTTP 2010					
		A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)	A(%)	R(%)	P(%)	F1(%)	FPR(%)	FNR(%)
XGBoost	-	96.229	92.421	96.854	94.407	0.546	14.61	92.133	92.133	93.148	92.087	0.200	15.533
GB	-	94.444	88.543	95.686	91.502	0.558	22.355	86.200	86.200	89.092	85.940	0.200	27.400
RF	-	99.597	100	98.274	99.129	0.523	0	99.867	99.867	99.867	99.867	0.267	0
BC	SVML	98.380	95.649	97.240	96.438	0.808	4.351	96.300	96.300	96.301	96.300	3.933	3.467
BC	DT	98.636	95.768	98.239	96.988	0.511	4.232	97.967	97.967	97.987	97.966	1	3.067
BC	LR	96.741	87.904	97.650	92.521	0.629	12.096	95.200	95.200	95.481	95.193	0.867	8.733

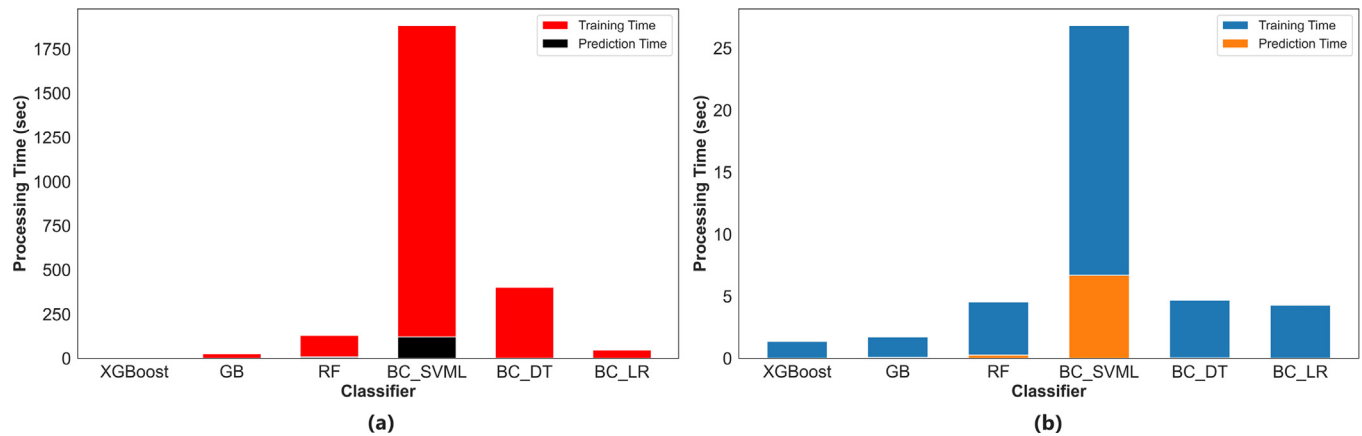


Fig. 6. Homogeneous ensemble training and prediction time measurements based on the (a) ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

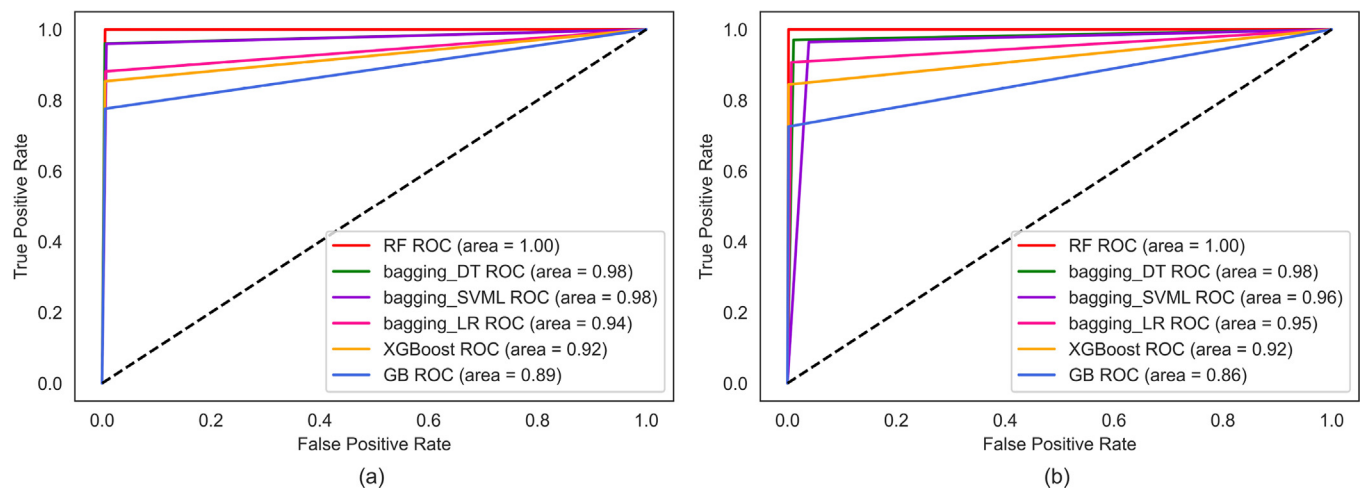


Fig. 7. ROC curve of homogeneous ensemble based on the (a) ECML/PKDD 2007 and (b) CSIC HTTP 2010 datasets.

Table 10

Summary of experimental results.

ML Techniques	Strengths	Weaknesses
NB	- Both MNB and BNB provide faster training and prediction times. - MNB is more accurate in classifying web attacks than BNB.	- BNB's low R value and high FNR make it inefficient for web-based assault detection. - Due to its low P and high FPR, MNB affects the availability of web applications.
KNN	- Good performance in terms of precision and FPR. - Needed the least training and prediction times.	- Poor detection performance in terms of R and FNR. - Weak ROC curve area of 89%
SVM	- SVM linear is more accurate for web application security than RBF and Sigmoid. - SVM linear provide faster training and prediction times than RBF and Sigmoid.	- Slow training and prediction times than other single classifiers.
DT	- Better performance in terms of A, R, P, FNR, and F1. - The area of the ROC curve is higher than 98%.	- Higher FPR compared to SVML and LR. - Slow training and prediction times compared to MNB, BNB, LR and KNN
LR	- Faster training and prediction times. - Good performance in terms of A, R, P, F1, and FPR - The area of the ROC curve is higher than 96%.	- Higher FNR compared to SVML and DT
Max Voting	- Higher A, P, and FPR than single classifiers. - Better performance in terms of A, R, P, F1, FPR, and area of the ROC curve than stacking.	- Slow training and prediction times compared to single classifier. - Higher FNR than stacking method.
Stacking	Stacking using SVML as final estimator provide good detection performance than stacking using DT and LR as a final estimator.	- Slow training and prediction times compared to Max voting and single classifiers. - Poor detection performance compared to single classifiers.
XGBoost	Faster training and prediction times than others.	- Poor detection performance compared to others in terms of A, R, P FNR, FPR and area of the ROC curve.
GB	Faster training and predictions times than others.	- Poor detection performance compared to others in terms of A, R, P FNR, FPR and area of the ROC curve.
RF	- More accurate for web application security compared to single classifiers, BC, boosting-based classifiers. - Good performance in terms of A, P, and FPR than single classifiers.	- Slow training and prediction times compared to single classifiers and boosting-based classifiers.
BC	- Good detection performance compared to boosting-based classifiers. - BC using DT as base models performed better than BC using SVML and LR.	- Slow training and prediction times compared to single classifiers, RF, and boosting-based classifiers

4.3. Discussion

Ensemble methods and single classifiers, two attack detection approaches frequently used for protecting web applications, were evaluated in this study. The performance evaluations of the single classifiers showed that SVM, DT, and LR obtained the highest performance in terms of A, R, P, F1, FPR, FNR, and area of the ROC values while maintaining a reasonable amount of time for training and prediction. On the other hand, the KNN, MNB, and BNB classifiers had the fastest TT and PT but poor detection performance, which means that using these classifiers for web application security is a matter of choosing between processing time and performance. Moreover, the performance evaluation of the heterogeneous ensembles showed that the Max voting method outperformed the stacking method in terms of A, R, P, F1, FPR, FNR, TT, and PT. In addition, the performance of the stacking approach is affected by the FE utilized. When the authors used the SVM and LR classifiers as FE, better results were achieved in terms of A, R, P, F1, FPR, FNR, TT, PT, and area of the ROC values than when they utilized the DT classifier. Subsequently, the performance evaluation of homogeneous ensembles demonstrated that the bagging-based classifier, practically RF, performed significantly better than the others in terms of A, R, P, F1, FPR, FNR, and area of the ROC curve. However, when it came to how long it took to train

and make predictions, boosting-based classifiers were better than bagging-based classifiers.

4.3.1. Comparison between single classifiers, heterogeneous and homogeneous ensembles in terms of A, R, P, FPR, FNR, and ROC curve value

When comparing the performance of single classifiers to that of the RF classifier, the authors observed that the RF outperformed SVM, LR, and DT in terms of A, R, P, F1, FPR, FNR, and ROC curve value (see Fig. 8 and Fig. 9). Additionally, single classifiers performed much better than BC in terms of A, R, P, and the value of the ROC curve. However, the authors found that when FPR was taken into consideration, BC was the algorithm that performed the best (see Fig. 8 and Fig. 9). In contrast to bagging-based classifiers, single classifiers outperformed boosting-based classifiers in terms of A, R, P, FNR, and ROC curve value, but in terms of FPR, boosting-based classifiers surpassed single classifiers (see Fig. 10 and Fig. 11). The comparison between Max voting and single classifiers revealed that the Max voting method is appropriate when accuracy is the most important element, whereas single classifiers in practically LR and DT classifiers can be employed when the FPR and FNR are the most important factors, respectively. In addition, single classifiers achieved better performance in terms of A, P,

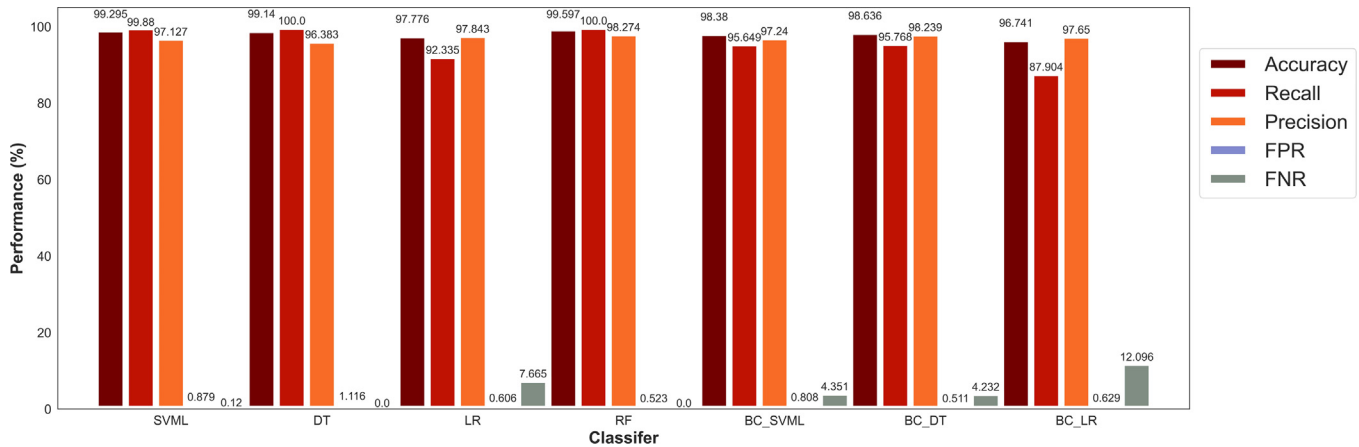


Fig. 8. Comparison between single classifiers and bagging-based classifiers based on the ECML/PKDD 2007 dataset.

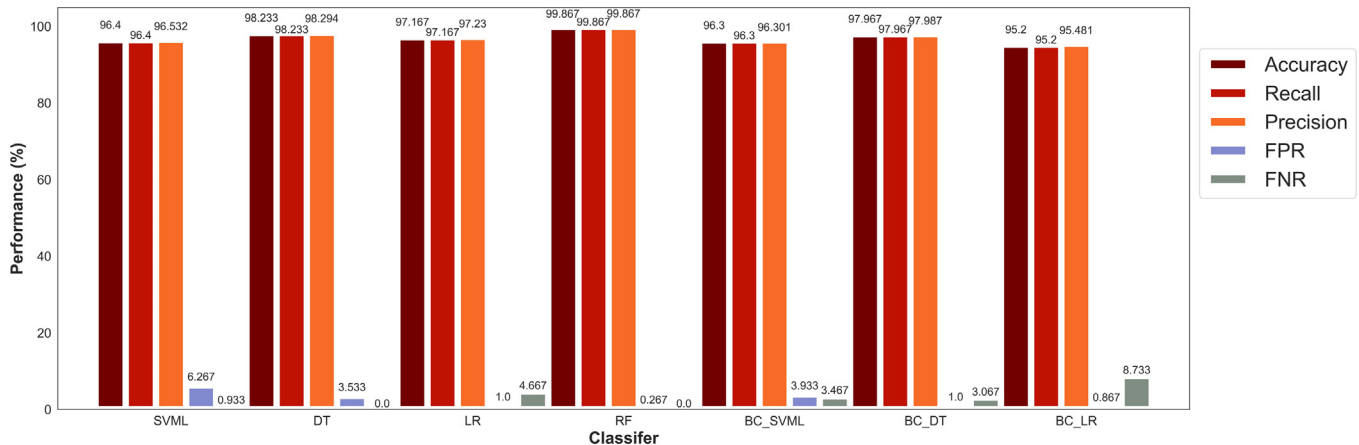


Fig. 9. Comparison between single classifiers and bagging-based classifiers based on the CSIC HTTP 2010 dataset.

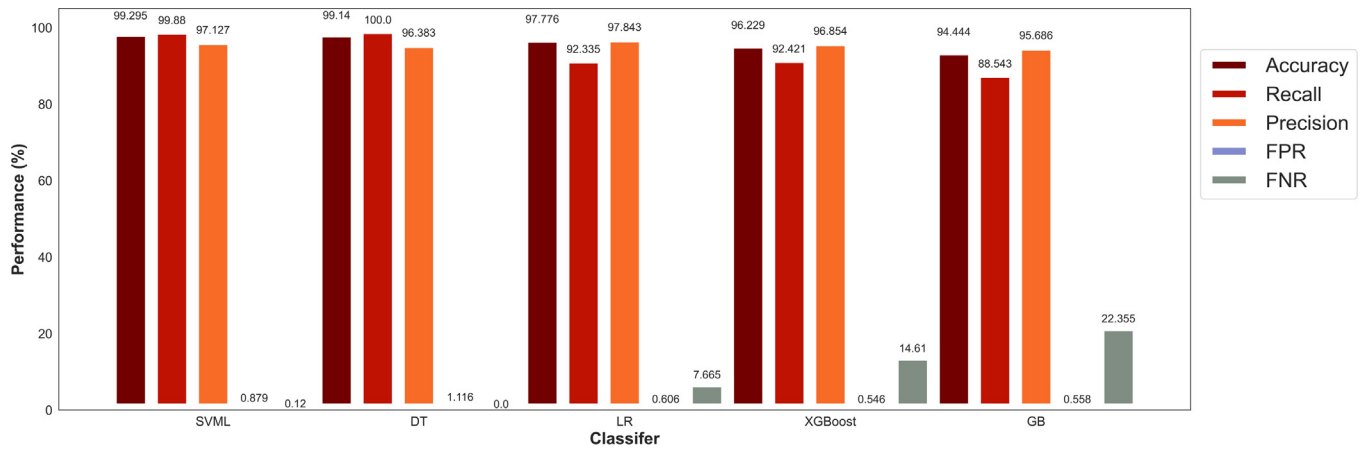


Fig. 10. Comparison between single classifiers and boosting-based classifiers based on the ECML/PKDD 2007 dataset.

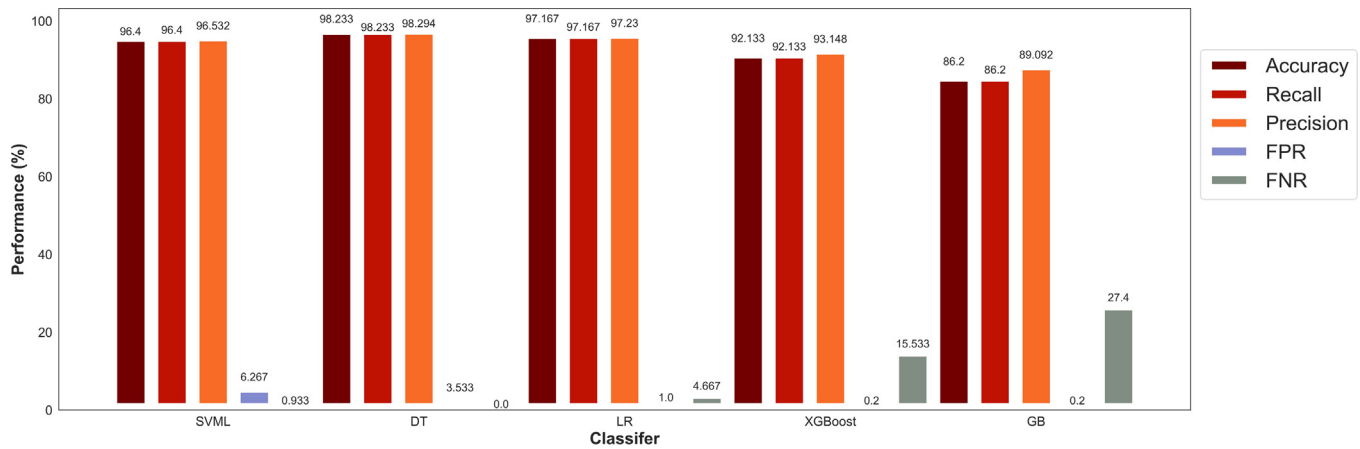


Fig. 11. Comparison between single classifiers and boosting-based classifiers based on the CSIC HTTP 2010 dataset.

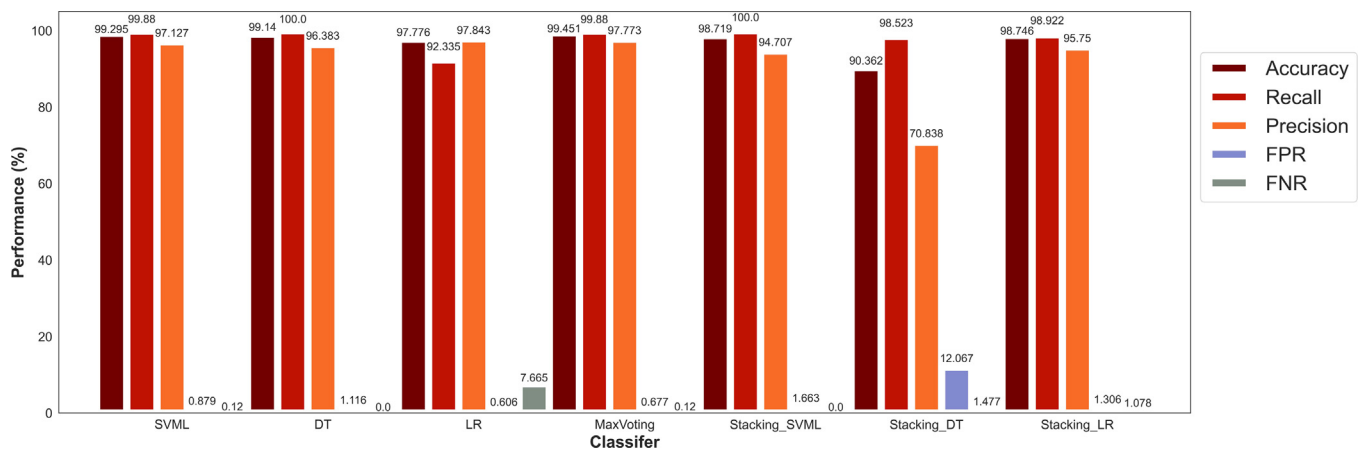


Fig. 12. Comparison between single classifiers and heterogeneous ensembles based on the ECML/PKDD 2007 dataset.

FPR, and the value of the ROC curve than stacking methods (see Fig. 12 and Fig. 13).

4.3.2. Comparison between single classifiers, heterogeneous and homogeneous ensembles in terms of TT and PT

The authors can clearly see that the ensemble methods needed the longest time to train their classification model and they are

more affected by the total number of data than a single classifier. It is also worth mentioning that the PT is very short than TT for the majority of the trained models which is measured in seconds. As can be seen from Fig. 14, bagging using SVML as BM and stacking methods took too much time to predict the unseen data than other classifiers. The results based on the ECML/PKDD 2007 dataset showed that staking using SVML(Stacking_SVML), DT(Stack-

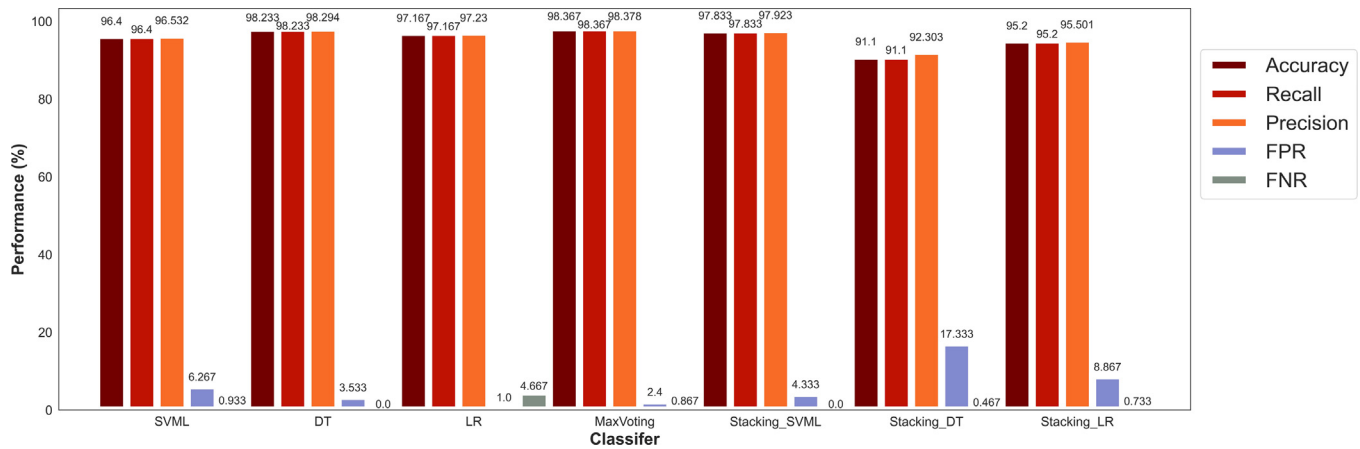


Fig. 13. Comparison between single classifiers and heterogeneous ensembles based on the CSIC HTTP 2010 dataset.

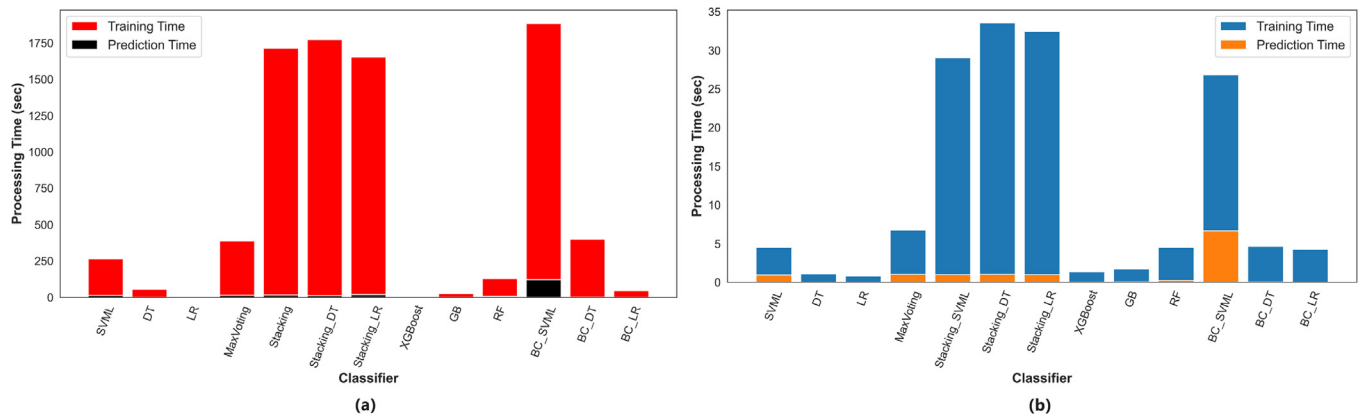


Fig. 14. Comparison between single classifiers heterogeneous and homogeneous ensembles in terms of training and prediction times based on the (a) ECML PKDD/2007 and (b) CSIC HTTP 2010 datasets.

ing_DT), and LR(Stacking_LR) as FE, and bagging using SVM (BC_SVML) as BM required long TT and PT with values of (1714.63s and 16.21s), (1772.88s and 12.04s), (1654.45s and 19.40s), and (1883.78s and 120.66s), respectively. Likewise, the CSIC HTTP 2010 dataset results revealed that Stacking_SVML, Stacking_DT, Stacking_LR, and BC_SVML required long TT and PT with values of (29.07s and 1s), (33.56s and 1.03s), (32.45s and 1s), and (26.83s and 6.68s), respectively.

4.3.3. Research questions answers

Based on the results of the experiments, the authors provide answers to the aforementioned research questions.

• **RQ1: Are the ensemble methods suitable for web-based attacks detection?** Concerning ensemble learning methods, security professionals and researchers must choose between high detection performance in terms of accuracy, precision, and FPR or high performance in terms of recall, FNR, area of the ROC curve, training, and prediction times. For this reason, the authors cannot assert that ensemble learning methods are 100% suited for web application security because it depends on the application of interest. One can choose a lower FNR rather than a higher FPR, which is also true for the other metrics.

• **RQ2: Which ensemble method is better suited for web-based attack detection?** When comparing the performance evaluation of the heterogeneous and homogeneous ensembles, the authors observed that homogeneous ensembles, particularly RF, are more suited for web-based attack detection because they per-

formed better in terms of accuracy, recall, precision, F-value, FPR, FNR, area of the ROC curve.

• **RQ3: How effective are ensemble-learning methods compared to single classification techniques for web-based attack detection?**

The experimental results of this study indicated that ensemble learning methods do not always outperform single classifiers in web-based attack detection, as is common in the literature. The authors discovered that both have advantages and disadvantages. Ensemble approaches exhibited higher accuracy, precision, and low FPR than single classifiers. However, in terms of recall, FNR, training, and prediction times, single classifiers performed better. To ensure the security of a web application, it is necessary to use a web-based attack detection system (WADS) that can present a good detection performance in terms of accuracy, precision, recall, FPR, and FNR with acceptable training and prediction times. Because using a WADS with good detection performance but slow training and prediction times will affect the web application's availability, and the same is true for all assessment metrics. A WADS with a low FPR and a higher FNR indicates that it cannot accurately detect malicious activity.

5. Challenges of current ML-WSS

With the increased use and rapid growth of the internet, ML-based Web Security Systems (ML-WSS) have become one of the

most preferred solutions for web application security. Many researchers and practitioners have developed various ML-based techniques for detecting web-based threats. However, the present suggested ML-WASS faces several challenges. In this paper, the authors categorize the challenges of the current ML-WSS into seven main categories:

- **ML-WSS Performance Evaluation:** In the evaluation phase of ML-WSS, it is crucial to employ appropriate measures that reveal the advantages and disadvantages of the proposed approach. The use of accuracy, precision, or FPR alone is insufficient for evaluating the performance of an ML-WSS. It is essential to implement all of the evaluation metrics presented in this study.
- **ML-WSS Performance Monitoring:** As attackers enhance their techniques, the nature of the threat landscape evolves rapidly (Sadqi and Maleh, 2022). Therefore, it is crucial to regularly monitor web-based attack detection and prevention systems for signs of performance degradation and to take corrective action as necessary (Bitton et al., 2021).
- **Real-Time ML-WSS:** The researchers have to consider the prevention side, it is crucial to develop a real-time ML-based security system that can detect and prevent web-based attacks before they damage the web application. Also, the time needed for the ML algorithm to analyze an HTTP request is crucial because an ML algorithm that takes a long time will affect the web application's availability.
- **Target Attacks Types:** Most of the proposed detection systems focus on the detection of a specific type of web-based attacks such as SQLI, or XSS (Zhou and Wang, 2019; Khan et al., 2017; Tripathy et al., 2020). This highlights the need for an ML-WSS that can detect and prevent various types of web-based attacks.
- **Lack of Available Web-based Attack Datasets:** One of the biggest problems in the ML-based web security field is the need for a public, recent, and large web-based attack dataset to train and validate ML-WSS (Riera et al., 2022). The performance of any ML-based system is highly dependent on the quality and quantity of the employed dataset. Consequently, the biases and constraints of the datasets utilized will directly impact the reliability of the model's predictions (Gümüşbaş et al., 2020). The publicly available benchmarking datasets, such as the KDD family and UNSW-NB 15, are not suitable for web-based attack detection because they were generated for network-based attack detection. The CIC-IDS 2017 and CSE-CIC-IDS 2018 are the most recent datasets that contain both network-based attacks and web-based attacks. However, these datasets have only 2180 and 928 web-based attack instances, respectively. Also, they only include three kinds of web attacks: SQLI, XSS, and brute force web. This means that these datasets are not suitable for training and evaluating ML-WSS. Consequently, the creation of novel public, recent, and reliable web-based attack datasets is one of the most unresolved problems in ML-based web security.
- **Lack of Explainability/Interpretability:** Unfortunately, the primary goal of cybersecurity researchers is to develop an effective ML-WSS using ensemble learning methods or single-classifier approaches with high detection performance, without giving much attention to the explainable and interpretable side of the ML-WSS (Adadi and Berrada, 2018). As a consequence of the black-box nature of ML algorithms, the experts need to use eXplainable AI (XAI) tools to trust and understand the mechanism by which these algorithms work and then avoid false predictions that may harm the web application's confidentiality, integrity, and availability (Adadi and Berrada, 2018; Kuppa and Le-Khac, 2021; Mahbooba et al., 2021).

- **Vulnerability to Adversarial Attacks:** Recent research has shown that systems that rely on ML algorithms are vulnerable to a novel category of attacks known as adversarial attacks (Bitton et al., 2021; Jmila and Khandedher, 2022). The main goal of the adversary, in this situation, is to alter the ML-WSS's ability to make the right classification and damage the web application. Consequently, the researchers have to evaluate the effectiveness of their ML-WSS against both traditional web-based attacks and adversarial attacks targeting the ML algorithm used to ensure web application security.

6. Conclusion & future work

While the development of Industry 5.0 enablers like AI, big data analytics, and Edge Computing permits a paradigm change in manufacturing, cyberattacks represent serious challenges to the industry. In that context, several ML-based solutions have been proposed to detect cyberattacks that take advantage of web-based threats and execute assaults that might impair essential Industry 5.0 capabilities. However, an extensive empirical evaluation of ensemble learning compared to single-learner techniques, applied to web application security is still currently lacking. In order to fill this research gap, this research evaluated the effectiveness of ensemble methods in comparison to single classifiers for web-based attack detection, using the commonly used datasets, namely ECML/PKDD 2007 and CSIC 2010 HTTP. First, the authors carefully evaluated the most popular single classifiers for web-based attack detection. Second, ensemble methods designed specially for web application security are compared to single classifiers. In particular, both homogeneous and heterogeneous ensemble classifiers assessment are considered in the experiments. The results of this study demonstrated that the choice of single classifier or ensemble methods for web application security depends on the application of interest. One has the option of selecting a lower FNR over FPR. This holds true for all of the other assessment measures as well. The authors believe that findings from this study will aid security practitioners and researchers in determining the best learning-based solution for web application security. This research is limited in that it only considers traditional web-based attack detection. In the future, the authors will present a unique machine learning-based attack detection system for securing Industry 5.0 applications against both traditional and adversarial web-based attacks and propose an up-to-date web-based attack dataset.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Abba, S.I., Benaafi, M., Usman, A.G., Aljundi, I.H., 2022. Inverse groundwater salinization modeling in a sandstone's aquifer using stand-alone models with an improved non-linear ensemble machine learning technique. *J. King Saud Univ.-Comput. Informat. Sci.* 34 (10), 8162–8175. <https://doi.org/10.1016/j.jksuci.2022.08.002>.
- Aburomman, A.A., Reaz, M.B.I., 2016. A novel SVM-kNN-PSO ensemble method for intrusion detection system. *Appl. Soft Comput.* 38, 360–372. <https://doi.org/10.1016/j.asoc.2015.10.011>.
- Aburomman, A.A., Reaz, M.B.I., 2017. A survey of intrusion detection systems based on ensemble and hybrid classifiers. *Comput. Sec.* 65, 135–152. <https://doi.org/10.1016/j.cose.2016.11.004>.
- Adadi, A., Berrada, M., 2018. Peeking inside the black-box: a survey on explainable artificial intelligence (XAI). *IEEE Access* 6, 52138–52160. <https://doi.org/10.1109/ACCESS.2018.2870052>.

- Alaoui, R.L., Nfaoui, E.H., 2022. Deep learning for vulnerability and attack detection on web applications: a systematic literature review. *Future Internet* 14 (4), 118. <https://doi.org/10.3390/fi14040118>.
- Alsaedi, A., Alhuzali, A., Bamasag, O., 2022. Effective and scalable black-box fuzzing approach for modern web applications. *J. King Saud Univ.-Comput. Informat. Sci.* 34 (10), 10068–10078. <https://doi.org/10.1016/j.jksuci.2022.10.006>.
- Asif, M., Abbas, S., Khan, M.A., Fatima, A., Khan, M.A., Lee, S.W., 2021. MapReduce based intelligent model for intrusion detection using machine learning technique. *J. King Saud Univ.-Comput. Informat. Sci.* <https://doi.org/10.1016/j.jksuci.2021.12.008>.
- Belouch, M., Hadaj, S.E., 2017. Comparison of ensemble learning methods applied to network intrusion detection. In: *Proceedings of the Second International Conference on Internet of things, Data and Cloud Computing*, pp. 1–4. <https://doi.org/10.1145/3018896.3065830>.
- Bitton, R., Maman, N., Elovici, Y., Shabtai, A., Singh, I., Momiyama, S., 2021. Evaluating the Cybersecurity Risk of Real-World, Machine Learning Production Systems. *ACM Comput. Surv. (CSUR)*. <https://doi.org/10.1145/3559104>.
- Boopalan, P., Ramu, S.P., Pham, Q.V., Dev, K., Maddikunta, P.K.R., Gadekallu, T.R., Huynh-The, T., 2022. Fusion of federated learning and industrial Internet of Things: A survey. *Comput. Networks* 109048. <https://doi.org/10.1016/j.comnet.2022.109048>.
- Caruana, R., Niculescu-Mizil, A., 2006. An empirical comparison of supervised learning algorithms. In: *Proceedings of the 23rd International Conference on Machine Learning*, pp. 161–168. <https://doi.org/10.1145/1143844.1143865>.
- Catillo, M., Pecchia, A., Rak, M., Villano, U., 2021. Demystifying the role of public intrusion datasets: a replication study of DoS network traffic data. *Comput. Sec.* 108, 102341. <https://doi.org/10.1016/j.cose.2021.102341>.
- Chan, G.Y., Lee, C.S., Heng, S.H., 2013. Discovering fuzzy association rule patterns and increasing sensitivity analysis of XML-related attacks. *J. Network Comput. Appl.* 36 (2), 829–842. <https://doi.org/10.1016/j.jnca.2012.11.006>.
- Desmet, L., Piessens, F., Joosen, W., Verbaeten, P., 2006. Bridging the gap between web application firewalls and web applications. In: *Proceedings of the Fourth ACM Workshop on Formal Methods in Security*, pp. 67–77. <https://doi.org/10.1145/1180337.1180344>.
- Gaba, G.S., Kumar, G., Monga, H., Kim, T.H., Liyanage, M., Kumar, P., 2020. Robust and lightweight key exchange (LKE) protocol for industry 4.0. *IEEE Access* 8, 132808–132824. <https://doi.org/10.1109/ACCESS.2020.3010302>.
- Ghaffarian, S.M., Shahriari, H.R., 2017. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. *ACM Comput. Surv. (CSUR)* 50 (4), 1–36. <https://doi.org/10.1145/3092566>.
- Giménez, C.T., Villegas, A.P., Marañón, G. Á., 2010. HTTP data set CSIC 2010. Information Security Institute of CSIC (Spanish Research National Council), 64. <https://www.isi.csic.es/dataset/>.
- Gümüşbaş, D., Yıldırım, T., Genovesi, A., Scotti, F., 2020. A comprehensive survey of databases and deep learning methods for cybersecurity and intrusion detection systems. *IEEE Syst. J.* 15 (2), 1717–1731. <https://doi.org/10.1109/JSYST.2020.2992966>.
- Gupta, D., Rani, R., 2020. Improving malware detection using big data and ensemble learning. *Comput. Electr. Eng.* 86, 106729. <https://doi.org/10.1016/j.compeleceng.2020.106729>.
- Hasan, T., Malik, J., Bibi, I., Khan, W.U., Al-Wesabi, F.N., Dev, K., Huang, G., 2022. Securing industrial internet of things against botnet attacks using hybrid deep learning approach. *IEEE Trans. Network Sci. Eng.* <https://doi.org/10.1109/TNSE.2022.3168533>.
- Jemal, I., Haddar, M.A., Cheikhrouhou, O., Mahfoudhi, A., 2021. Performance evaluation of Convolutional Neural Network for web security. *Comput. Commun.* 175, 58–67. <https://doi.org/10.1016/j.comcom.2021.04.029>.
- Jmila, H., Khandedher, M.I., 2022. Adversarial machine learning for network intrusion detection: A comparative study. *Comput. Networks* 109073. <https://doi.org/10.1016/j.comnet.2022.109073>.
- Kasongo, S.M., Yanxia, S., 2020. Performance Analysis of Intrusion Detection Systems Using a Feature Selection Method on the UNSW-NB15 Dataset. *J. Big Data* 7 (1), 1–20. <https://doi.org/10.1186/s40537-020-00379-6>.
- Khan, N., Abdullah, J., Khan, A.S., 2017. Defending malicious script attacks using machine learning classifiers. *Wireless Commun. Mobile Comput.* 2017. <https://doi.org/10.1155/2017/5360472>.
- Khan, S., Kifayat, K., Kashif Bashir, A., Gurtov, A., Hassan, M., 2021. Intelligent intrusion detection system in smart grid using computational intelligence and machine learning. *Trans. Emerg. Telecommun. Technol.* 32 (6), e4062. <https://doi.org/10.1002/ett.4062>.
- Khan, S., Thorn, J., Wahlgren, A., Gurtov, A., 2021. Intrusion Detection in Automatic Dependent Surveillance-Broadcast (ADS-B) with Machine Learning. In: *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*, IEEE, pp. 1–10. <https://doi.org/10.1109/DASC52595.2021.9594431>.
- Kozik, R., Choraś, M., Renk, R., Hołubowicz, W., 2015. A proposal of algorithm for web applications cyber attack detection. In: *IFIP International Conference on Computer Information Systems and Industrial Management*. Springer, pp. 680–687. https://doi.org/10.1007/978-3-662-45237-0_61.
- Kumar, M., Hanumanthappa, M., Kumar, T.S., 2012. Intrusion Detection System using decision tree algorithm. In: *2012 IEEE 14th International Conference on Communication Technology*, IEEE Xplore, pp. 629–634. <https://doi.org/10.1109/ICCT.2012.6511281>.
- Kumar, D.P., Amgoth, T., Annarapu, C.S.R., 2019. Machine learning algorithms for wireless sensor networks: A survey. *Informat. Fusion* 49, 1–25. <https://doi.org/10.1016/j.inffus.2018.09.013>.
- Kumar, P., Kumar, R., Garg, S., Kaur, K., Zhang, Y., Guizani, M., 2022a. A Secure Data Dissemination Scheme for IoT-Based e-Health Systems using AI and Blockchain. In: *GLOBECOM 2022–2022 IEEE Global Communications Conference*, IEEE, pp. 1397–1403. <https://doi.org/10.1109/GLOBECOM48099.2022.10000801>.
- Kumar, R., Kumar, P., Aloqaily, M., Aljuhani, A., 2022b. Deep Learning-based Blockchain for Secure Zero Touch Networks. *IEEE Commun. Mag.* <https://doi.org/10.1109/MCOM.001.2200294>.
- Kumar, R., Aljuhani, A., Kumar, P., Kumar, A., Franklin, A., Jolfaei, A., 2022c. Blockchain-enabled secure communication for unmanned aerial vehicle (UAV) networks. In: *Proceedings of the 5th International ACM Mobicom Workshop on Drone Assisted Wireless Communications for 5G and Beyond*, ACM, pp. 37–42. <https://doi.org/10.1145/3555661.3560861>.
- Kuppa, A., Le-Khac, N.A., 2021. Adversarial xai methods in cybersecurity. *IEEE Trans. Inf. Forensics Secur.* 16, 4924–4938. <https://doi.org/10.1109/TIFS.2021.3117075>.
- Leng, J., Sha, W., Wang, B., Zheng, P., Zhuang, C., Liu, Q., Wang, L., 2022. Industry 5.0: Prospect and retrospect. *J. Manuf. Syst.* 65, 279–295. <https://doi.org/10.1016/j.jmsys.2022.09.017>.
- Liao, J.H., Lin, C.H.R., Lin, Y.C., Tung, K.Y., 2013. Intrusion detection system: A comprehensive review. *J. Network Comput. Appl.* 36 (1), 16–24. <https://doi.org/10.1016/j.jnca.2012.09.004>.
- Lower, N., Zhan, F., 2020. A study of ensemble methods for cyber security. In: *10th Annual Computing and Communication Workshop and Conference (CCWC)*, IEEE Xplore, pp. 1001–1009. <https://doi.org/10.1109/CCWC47524.2020.9031256>.
- Luo, C., Tan, Z., Min, G., Gan, J., Shi, W., Tian, Z., 2020. A novel web attack detection system for internet of things via ensemble classification. *IEEE Trans. Ind. Informat.* 17 (8), 5810–5818. <https://doi.org/10.1109/TII.2020.3038761>.
- Maddikunta, P.K.R., Pham, Q.V., Prabadevi, B., Deepa, N., Dev, K., Gadekallu, T.R., Liyanage, M., 2022. Industry 5.0: A survey on enabling technologies and potential applications. *J. Ind. Informat. Integrat.* 26, 100257. <https://doi.org/10.1016/j.jii.2021.100257>.
- Maddikunta, P.K.R., Pham, Q.V., Prabadevi, B., Deepa, N., Dev, K., Gadekallu, T.R., Liyanage, M., 2022. Industry 5.0: A survey on enabling technologies and potential applications. *J. Ind. Informat. Integrat.* 26, 100257. <https://doi.org/10.1016/j.jii.2021.100257>.
- Mahbooba, B., Timilsina, M., Sahal, R., Serrano, M., 2021. Explainable artificial intelligence (XAI) to enhance trust management in intrusion detection systems using decision tree model. *Complexity* 2021. <https://doi.org/10.1155/2021/6634811>.
- Malowidzki, M., Berezinski, P., Mazur, M., 2015. Network intrusion detection: Half a kingdom for a good dataset. In: *Proceedings of NATO STO SAS-139 Workshop*.
- Maseer, Z.K., Yusof, R., Bahaman, N., Mostafa, S.A., Foozy, C.F.M., 2021. Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset. *IEEE Access* 9, 22351–22370. <https://doi.org/10.1109/ACCESS.2021.3056614>.
- Masud, M., Alazab, M., Choudhary, K., Gaba, G.S., 2021. 3P-SAKE: privacy-preserving and physically secured authenticated key establishment protocol for wireless industrial networks. *Comput. Commun.* 175, 82–90. <https://doi.org/10.1016/j.comcom.2021.04.021>.
- Mereani, F.A., Howe, J.M., 2018. Detecting cross-site scripting attacks using machine learning. In: *International Conference on Advanced Machine Learning Technologies and Applications*. Springer, pp. 200–210. https://doi.org/10.1007/978-3-319-74690-6_20.
- Mokhtari, S., Abbaspour, A., Yen, K.K., Sargolzaei, A., 2021. A machine learning approach for anomaly detection in industrial control systems based on measurement data. *Electronics* 10 (4), 407. <https://doi.org/10.3390/electronics10040407>.
- Opitz, D., Maclin, R., 1999. Popular ensemble methods: An empirical study. *J. Artif. Intell. Res.* 11, 169–198. <https://doi.org/10.1613/jair.614>.
- Oumaima, C., Abdeslam, R., Yassine, S., Abderrazek, F., 2021. Experimental Study on the Effectiveness of Machine Learning Methods in Web Intrusion Detection. In: *The International Conference on Information, Communication & Cybersecurity*, Springer, pp. 486–494. https://doi.org/10.1007/978-3-030-91738-8_44.
- Chedy Raissi, Johan Brissaud, Gérard Dray, Pascal Poncelet, Mathieu Roche, et al, 2007. Web Analyzing Traffic Challenge: Description and Results. *ECML PKDD 2007 Discovery Challenge*, 2007, Warsaw, Poland.
- Riera, T.S., Higuera, J.B., Higuera, J.B., Herraiz, J.M., Montalvo, J.S., 2022. A new multi-label dataset for Web attacks CAPEC classification using machine learning techniques. *Comput. Sec.* 120, 102788. <https://doi.org/10.1016/j.cose.2022.102788>.
- Sadqi, Y., Maleh, Y., 2022. A systematic review and taxonomy of web applications threats. *Informat. Sec. J.: A Global Perspect.* 31, 1–27. <https://doi.org/10.1080/19393555.2020.1853855>.
- Sadqi, Y., Mekkaoui, M., 2021. Design Challenges and Assessment of Modern Web Applications Intrusion Detection and Prevention Systems (IDPS). In: *The Proceedings of the Third International Conference on Smart City Applications*, Springer, pp. 1087–1104. https://doi.org/10.1007/978-3-030-66840-2_83.
- Sagi, O., Rokach, L., 2018. Ensemble learning: A survey. *Wiley Interdiscip. Rev.: Data Min. Knowledge Discovery* 8 (4), e1249. <https://doi.org/10.1002/widm.1249>.
- Schapire, R.E., 1999. A brief introduction to boosting. In: *The Sixteenth International Joint Conference on Artificial Intelligence*, IJCAI, 99, pp. 1401–1406. Available: <https://dl.acm.org/doi/10.5555/1624312.1624417>.
- Schmitt, I., Schinzel, S., 2012. WAFFle: fingerprinting filter rules of web application firewalls. In: *6th USENIX Workshop on Offensive Technologies*, WOOT 2012, pp. 34–40.

- Shahid, W.B., Aslam, B., Abbas, H., Khalid, S.B., Afzal, H., 2022. An enhanced deep learning based framework for web attacks detection, mitigation and attacker profiling. *J. Network Comput. Appl.* 198, 103270. <https://doi.org/10.1016/j.jnca.2021.103270>.
- Singh, P., Gaba, G.S., Kaur, A., Hedabou, M., Gurtov, A., 2022. Dew-cloud-based hierarchical federated learning for intrusion detection in IoMT. *IEEE J. Biomed. Health Informa.* <https://doi.org/10.1109/JBHI.2022.3186250>.
- Smitha, R., Hareesha, K.S., Kundapur, P.P., 2019. A machine learning approach for web intrusion detection: MAMLS perspective. *Soft Comput. Signal Process.* 119–133, https://doi.org/10.1007/978-981-13-3600-3_12.
- Sommer, R., Paxson, V., 2010. Outside the closed world: On using machine learning for network intrusion detection. In: *IEEE Symposium on Security and Privacy*, pp. 305–316. <https://doi.org/10.1109/SP.2010.25>.
- Tama, B.A., Lim, S., 2021. Ensemble learning for intrusion detection systems: A systematic mapping study and cross-benchmark evaluation. *Comput. Sci. Rev.* 39, 100357. <https://doi.org/10.1016/j.cosrev.2020.100357>.
- Tama, B.A., Rhee, H.K., 2017. An extensive empirical evaluation of classifier ensembles for intrusion detection task. *Int. J. Comput. Syst. Sci. Eng.* 32 (2), 149–158.
- Tama, B.A., Nkenyereye, L., Islam, S.R., Kwak, K.S., 2020. An enhanced anomaly detection in web traffic using a stack of classifier ensemble. *IEEE Access*, 8, 24120–24134, <https://doi.org/10.1109/ACCESS.2020.2969428>.
- Tekerek, A., 2021. A novel architecture for web-based attack detection using convolutional neural network. *Comput. Sec.* 100, 102096. <https://doi.org/10.1016/j.cose.2020.102096>.
- Tian, Z., Luo, C., Qiu, J., Du, X., Guizani, M., 2019. A distributed deep learning system for web attack detection on edge devices. *IEEE Trans. Industr. Inf.* 16 (3), 1963–1971. <https://doi.org/10.1109/TII.2019.293877>.
- Tripathy, D., Gohil, R., Halabi, T., 2020. Detecting SQL injection attacks in cloud SaaS using machine learning. In: *EEE 6th Intl Conference on Big Data Security on Cloud (BigDataSecurity)*, *IEEE Intl Conference on High Performance and Smart Computing, (HPSC)* and *IEEE Intl Conference on Intelligent Data and Security (IDS)*, *IEEE Xplore*, pp. 145–150, <https://doi.org/10.1109/BigDataSecurity-HPSC-IDS49724.2020.00035>.
- Valentini, G., Masulli, F., 2002. Ensembles of learning machines. In: *Italian Workshop on Neural Nets*. Springer, pp. 3–20. https://doi.org/10.1007/3-540-45808-5_1.
- Web Application attacks datasets, GitLab, Last accessed 21 november 2022, Available: <https://gitlab.fing.edu.uy/gsi/web-application-attacks-datasets/-/tree/master>.
- Wu, D., Ren, A., Zhang, W., Fan, F., Liu, P., Fu, X., Terpenney, J., 2018. Cybersecurity for digital manufacturing. *J. Manuf. Syst.* 48, 3–12.
- Yang, Z., Liu, X., Li, T., Wu, D., Wang, J., Zhao, Y., Han, H., 2022. A systematic literature review of methods and datasets for anomaly-based network intrusion detection. *Comput. Sec.* 116, 102675. <https://doi.org/10.1016/j.cose.2022.102675>.
- Zeb, S., Mahmood, A., Khowaja, S.A., Dev, K., Hassan, S.A., Qureshi, N.M.F., Bellavista, P., 2022. Industry 5.0 is coming: A survey on intelligent nextG wireless networks as technological enablers. *arXiv preprint arXiv:2205.09084*.
- Zhou, Y., Wang, P., 2019. An ensemble learning approach for XSS attack detection with domain knowledge and threat intelligence. *Comput. Sec.* 82, 261–269. <https://doi.org/10.1016/j.cose.2018.12.016>.
- Zhou, Y., Cheng, G., Jiang, S., Dai, M., 2020. Building an efficient intrusion detection system based on feature selection and ensemble classifier. *Comput. Networks* 174, 107247. <https://doi.org/10.1016/j.comnet.2020.107247>.